

Wohlgenannt-Markus_IU14080395_DataEngineering_P1_S

Stream Processing Pipeline for Municipal Environmental Sensors

Konzeptionsphase

Markus Wohlgenannt

July 10, 2026

1. Data Source & Objective

The project aims to build a robust stream processing pipeline for environmental telemetry data. Since the municipality's actual IoT sensors are not yet deployed, the free Open-Meteo weather API serves as a stand-in. Five fixed coordinates simulate city-wide sensor stations. Every 10 seconds, a Python script polls metrics (temperature, humidity, wind speed). Each reading is formatted as a flat JSON object, mimicking an IoT gateway payload. This structure ensures that future metrics (e.g., CO2, fine dust) can be appended without breaking the pipeline. A local fallback simulator generates plausible payloads to ensure testability during API limits or network outages.

2. Operational Purpose & User Stories

The pipeline serves two distinct user groups with very different requirements regarding data access and reliability:

User Story 1: City Planners (Historical & Analytical Needs)

"As a city planner, I need access to months or years of aggregated environmental data to run regression models (e.g., mapping urban heat islands) and plan long-term infrastructure interventions."

Operationalization: Planners do not rely on day-by-day or per-second operations. They require a comprehensive, uncorrupted historical dataset. Their reliability depends on absolute data completeness over time. They will query the database using batch operations and MongoDB Aggregation Pipelines to extract daily, weekly, or monthly district averages for their models.

User Story 2: Citizens (Real-time & Safety Needs)

"As a vulnerable citizen (e.g., fragile elderly or asthmatic), I need an app that warns me of dangerous current conditions and explicitly tells me when the data is unreliable."

Operationalization: For citizens, reliability means *data currency and truthfulness*. If a sensor breaks or the network goes down, showing stale (but previously safe) data is dangerous. Reliability here is operationalized by implementing a staleness check: if the app's backend does not consume new Kafka messages for a specific station within a few minutes, the app must explicitly display a "Sensors Offline / Data Unavailable" warning. This empowers vulnerable users to stay safely indoors rather than trusting outdated data.

3. Technology Choice

The chosen stack (Kafka + MongoDB) directly bridges the gap between ingestion velocity and the diverse user requirements:

- **Apache Kafka (Ingestion & Pipeline Reliability):** Chosen over Spark Streaming as the project requires reliable buffering, not complex in-flight aggregation. Kafka's durable log ensures system reliability: messages persist until committed, acting as a resilient buffer. This guarantees that no data is lost during backend updates, satisfying the planners' need for a gapless historical archive, while delivering low-latency streams to the citizen app backend.
- **MongoDB (Storage & Querying):** MongoDB is selected for its schema-less document model, which directly mirrors the ingested JSON payloads. It allows the immediate integration of new sensor types without database migrations. To satisfy the city planners, MongoDB's native Time Series collections and Aggregation Framework will be utilized, allowing them to efficiently run complex analytical queries (grouping, averaging, regression inputs) over large historical datasets.

4. Implementation Plan

1. Provision a single-node Kafka (KRaft mode) and a MongoDB instance via `docker-compose.yml`.
2. Implement a Python producer polling Open-Meteo per station and publishing JSON to a `sensor-readings` topic, including a fallback simulator for outage testing.
3. Implement a Python consumer that acts as the backend: reading the topic, writing to MongoDB, and tracking data freshness to trigger "sensor broken/offline" states.
4. Containerize the Python scripts to create a portable, one-click Docker environment.
5. Publish the code and infrastructure setup to a public GitHub repository.